ASSIGNMENT 1

Algorithmic Efficiency & Recursion Toolkit (AERT) REPORT

Course: Data Structures
Course Code: ETCCDS202
Name: Palakshi Kapoor
Roll No: 2501730012
Section: A (BTECH CSE (AI & ML))

1. Stack ADT

An Abstract Data Type (ADT) defines what operations can be performed on data but does not specify how they are implemented internally.

In this assignment, I implemented a Stack ADT with the following operations:

- push(x)
- pop()
- peek()
- is_empty()
- size()

The stack follows the LIFO (Last In First Out) principle.

Why Stack Operations are O(1)

Operations like push, pop and peek work only on the top element of the stack. They do not depend on the size of the stack. Therefore:

Time Complexity = O(1)
Best, Average and Worst cases = O(1)

Usage of StackADT

In this assignment, I used StackADT to:

- Store moves in Tower of Hanoi
- Store mid indices visited in Recursive Binary Search

This demonstrates practical use of ADT.

2. Recursive Factorial

Factorial is defined as:

$n! = n \times (n - 1)!$
Base case: $0! = 1$

In the recursive implementation:

- If $n = 0 \rightarrow$ return 1
- Otherwise $\rightarrow$ return $n \times$ factorial$(n - 1)$

Time Complexity

$O(n)$

Because the function makes n recursive calls before reaching the base case.

Space Complexity

$O(n)$

Because n recursive calls are stored in the call stack.

Cases

Best Case = $O(n)$
Average Case = $O(n)$
Worst Case = $O(n)$

All cases are the same since factorial always performs n recursive calls.

3. Fibonacci

3.1 Naive Recursive Fibonacci

The recursive formula:

$fib(n) = fib(n - 1) + fib(n - 2)$

This version recalculates the same values multiple times. For example, fib(5) calculates fib(3) more than once.

Time Complexity

$O(2^n)$

Because each function call creates two more recursive calls, leading to exponential growth.

Space Complexity

O(n)

Because the recursion depth can go up to n.

Big-Ω and Big-Θ

Since the growth is exponential in all cases,
Time Complexity is also Θ(2^n).

This method becomes very slow for larger values like 20 or 30 due to repeated calculations.

3.2 Memoized Fibonacci (Dynamic Programming)

In memoization, previously calculated Fibonacci values are stored in a dictionary. Before computing fib(n), the function checks if the value already exists.

This avoids repeated calculations.

Time Complexity

O(n)

Because each Fibonacci value from 0 to n is calculated only once.

Space Complexity

O(n)

Because:

- n values are stored in the dictionary
- Recursion stack depth is at most n

Why This is Dynamic Programming

Memoization solves overlapping subproblems and stores their solutions. Dynamic Programming, which is used when a problem contains overlapping subproblems. Instead of solving the same subproblem multiple times, dynamic programming stores the results and reuses them whenever needed.This is the key idea of Dynamic Programming.

4. Tower of Hanoi

Tower of Hanoi is solved using recursion.

Base Case

If $n = 1$, move the disk directly from source to destination.

Recursive Idea

To move n disks:

1. Move $n - 1$ disks to helper rod
2. Move the largest disk to destination
3. Move $n - 1$ disks from helper rod to destination

Number of Moves

Recurrence relation:

$T(n) = 2T(n - 1) + 1$

Solving this gives:

Number of moves = $2^n - 1$

For $n = 3$, total moves = 7.

Time Complexity

$O(2^n)$

Because the number of operations grows exponentially.

Space Complexity

$O(n)$

Because recursion depth is n.

5. Recursive Binary Search

Binary Search works on a sorted array.

Steps:

1. Find middle element
2. If it equals key → return index
3. If key is smaller → search left half
4. If key is larger → search right half

Recurrence Relation

$T(n) = T(n/2) + O(1)$

Each step reduces the problem size by half.

Time Complexity

$O(\log n)$

Space Complexity (Recursive Version)

$O(\log n)$

Because recursive calls go up to log n depth.

Cases

Best Case: $O(1)$
(If element is found at middle)

Average Case: $O(\log n)$

Worst Case: $O(\log n)$
(If element is not present or at extreme end)

Binary Search follows the Divide and Conquer paradigm because it divides the problem into smaller parts and solves one part recursively.

6. Paradigm Reflection

Divide and Conquer
Divide and Conquer works by dividing a large problem into smaller subproblems of the same type, solving them independently, and combining their results to obtain the final solution.
Binary search works on a sorted array and repeatedly divides the search space into two halves. It compares the key with the middle element of the array. If the key is smaller, the algorithm searches in the left half; if it is larger, it searches in the right half. This process continues until the element is found or the search space becomes empty.

Real-life example: Searching a word in a dictionary.

Dynamic Programming

Dynamic Programming, which is used when a problem contains overlapping subproblems. Instead of solving the same subproblem multiple times, dynamic programming stores the results and reuses them whenever needed.

The memoized Fibonacci implementation demonstrates this concept. In the naive recursive Fibonacci method, many values are recomputed repeatedly, which leads to an exponential time complexity of $\mathbf{O(2^n)}$.

However, by using memoization, previously calculated Fibonacci values are stored in a dictionary. When the function is called again with the same input, the stored result is returned immediately. This reduces the time complexity to **O(n)**.

Real-life example: Finding shortest path in navigation systems.

Recursion

Recursion occurs when a function calls itself to solve a smaller version of the same problem. Every recursive algorithm must have a base case that stops the recursion and a recursive case that continues the process with smaller input values.

In the Tower of Hanoi problem, the task of moving multiple disks is broken down into smaller tasks of moving fewer disks. The algorithm first moves the top disks to an auxiliary rod, moves the largest disk to the destination, and finally moves the remaining disks onto the destination rod.

Real-life example: Opening nested folders inside folders.

Greedy

Greedy algorithms make the best possible choice at each step without reconsidering previous decisions.

Real-life example: Giving change using the largest denomination first.